Digital Forensics Lab Guide

Cybersecurity Masterclass · Digital Forensics Course

Oct 27, 2026 · @Joshua Moses

About this lab guide

This guide takes a student from an empty computer to a finished, defensible forensic report in twelve labs and a capstone. Every lab ties back to the principles of digital forensics, uses free or free-to-register tools, and produces a result the student can check against a known answer.

The key design choice is simple. Students build their own evidence. In Part 5 each student runs a seeding script on a disposable Windows virtual machine that plays out a small insider data theft. Because the student created the activity, every lab has a ground truth to measure against, and the instructor can tell at a glance whether a finding is real or a tool artifact.

Who this is for

- Students in the Cybersecurity Masterclass who have finished the virtualization lab guide and can build Windows and Linux virtual machines.
- Comfort with the Windows command prompt, PowerShell, and a Linux terminal is assumed. No prior forensic experience is needed.
- Instructors running the course in a classroom, or students working self-paced.

How the guide is organized

Part	What it covers	When you use it
1. Principles	The 12 principles, seven pillars and governing standards	Read before Lab 1, revisit before the capstone
2. Lab architecture	The virtual machines, networks and folder layout	Once, during setup
3. Software inventory	Every tool, what it does, where it comes from	Reference

Part	What it covers	When you use it
4. Install and configure	Step by step setup for each tool, with a check that proves it works	Once, during setup
5. Evidence scenario	Building the Case DF-2026-001 subject machine	Once, before Lab 1
6. Labs 1 to 12	Hands-on exercises, each with intended results and why	Weekly
Capstone	A full investigation and written report	Final assessment
Appendices	Forms, templates, command reference, rubric	Throughout

How each lab is laid out

Every lab follows the same pattern so students know where to look.

1. **Objectives**: what you will be able to do when finished.
2. **Principles reinforced**: which of the 12 principles the lab exercises.
3. **Time and prerequisites**: what must already be done.
4. **Procedure**: numbered steps with exact commands and menu paths.
5. **Intended result**: what you should see.
6. **Why this result matters**: the forensic reasoning behind it.
7. **Deliverables**: what you hand in.
8. **Check your understanding**: three or four questions.

Ground rules

These rules apply to every lab. Breaking one is an automatic fail on that lab.

- **Lab evidence only.** Examine only the virtual machines, images, captures and memory dumps you created for this course. Never image, capture or examine a device you do not own or have written permission to examine. Principle 9 (Legal Authority) starts here.
- **Isolate the lab.** The subject machine and the forensic workstations run on a host-only network. The only exceptions are tool installation and the 30-minute seeding window in Part 5.

- **Document as you go.** Notes written after the fact are reconstruction. Every case notes entry carries the time it was written.
- **Never work on the original.** Hash it, copy it, verify the copy, then work on the copy.
- **Treat tool output as a lead, not a fact.** If a tool says something happened, find the artifact that shows it.

Part 1: Principles of Digital Forensics

Digital forensics is the scientific process of identifying, preserving, collecting, examining, analyzing and presenting digital evidence so that it stays intact, authentic, reliable and admissible. Recovering a deleted file is a skill. Proving how you recovered it, that you changed nothing, and that someone else would get the same answer is the discipline.

Every lab in this guide is built to exercise these principles. Read this part before Lab 1 and come back to it before you write the capstone report.

1.1 The 12 fundamental principles

#	Principle	What it means	What it looks like in this lab guide	Labs
1	Evidence Integrity	Evidence stays unaltered, or every change is explained and documented.	You hash every item at acquisition and again before every analysis session.	2, 3, 4, all
2	Chain of Custody	Every transfer, possession and access is traceable.	A custody form (Appendix A) travels with each evidence item from Lab 1 onward.	1, 3, 4
3	Forensic Soundness	Methods are reliable, validated and suited to the evidence.	You validate each tool against a known answer before trusting it.	2, 4, 5
4	Repeatability	The same examiner gets the same result under the same conditions.	You image the same drive twice and compare hashes.	2, 4

#	Principle	What it means	What it looks like in this lab guide	Labs
5	Reproducibility	A different examiner can verify the findings with your documented method.	Lab partners swap images and reproduce one finding from each other's notes.	5, 9, capstone
6	Documentation	Tools, versions, commands, timestamps, actions and findings are recorded.	Every lab has a case notes deliverable in the Appendix C format.	All
7	Order of Volatility	Collect what disappears fastest first, weighed against the risk of collecting it.	Memory and live state are captured before the disk is imaged.	3, 7
8	Least Intrusion	Make as few changes to the system and evidence as possible.	You measure the footprint your own live response leaves behind.	3, 11
9	Legal Authority	Every search and acquisition has consent, authorization or a warrant.	Each case opens with a signed authorization record.	1, capstone
10	Objectivity	Separate observed facts from interpretations and assumptions.	Findings are tagged FACT, INTERPRETATION or HYPOTHESIS.	9, 10, capstone
11	Evidence Corroboration	Important conclusions rest on more than one independent artifact.	No key finding is accepted until two artifact sources agree.	6, 9, capstone
12	Defensible Reporting	Findings are accurate, clear, qualified and backed by evidence.	The capstone report is reviewed by a peer before submission.	Capstone

A forensic tool's output is not a forensic fact. A tool parses bytes and applies rules someone wrote. When it reports that a USB drive was connected at 14:32, the fact is the registry value or log entry it read. Your job is to know which artifact that was, what it really

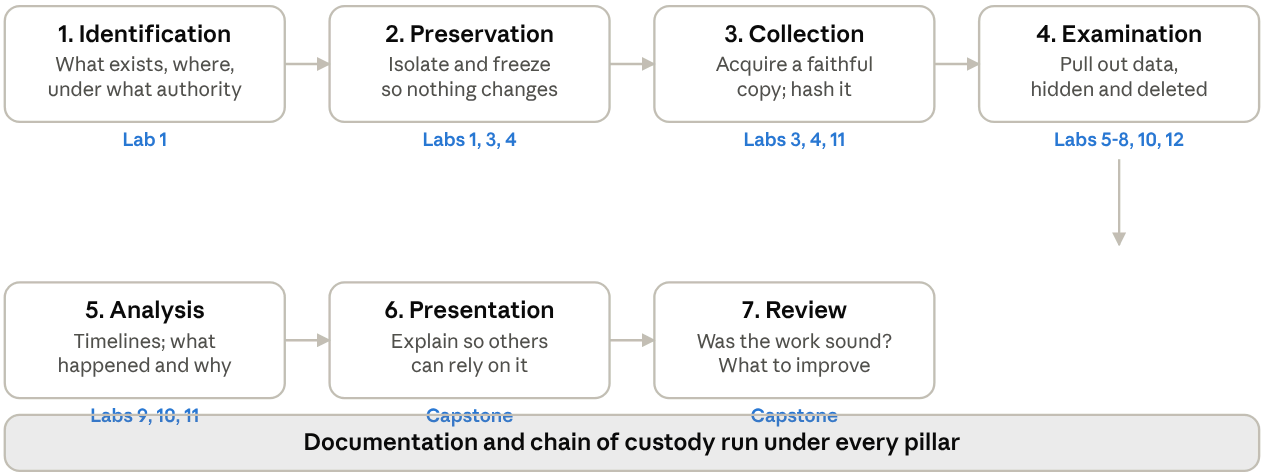
records, and whether a second source agrees.

1.2 The seven operational pillars

The pillars describe the life of an investigation from first call to final report. They combine common practice from NIST SP 800-86, ISO/IEC 27037 and ISO/IEC 27043 into one teaching model. No single standard prescribes exactly seven pillars, so treat this as a map, not a rulebook.

Pillar	Core question	Typical outputs	Labs
1. Identification	What evidence exists, where is it, and what authority covers it?	Scope statement, evidence inventory, authorization	1
2. Preservation	How do we stop it from changing or vanishing?	Isolation, write blocking, snapshots, legal hold	1, 3, 4
3. Collection	How do we acquire a faithful copy?	Images, memory dumps, captures, hash values, custody forms	3, 4, 11
4. Examination	What data is inside, including hidden or deleted data?	Parsed artifacts, recovered files, extracted metadata	5, 6, 7, 8, 10, 12
5. Analysis	What does the data mean, and what happened?	Timelines, correlations, tested hypotheses	9, 10, 11
6. Presentation	How do we explain it so a non-expert can rely on it?	Written report, exhibits, testimony	Capstone
7. Review	Was the work sound, and what should change next time?	Peer review notes, lessons learned, method updates	Capstone

Seven pillars of an investigation, documentation underneath all of them



the seven operational pillars, with documentation underneath all of them

Documentation is not a pillar on its own because it runs underneath all seven. If a step is not in your notes, the next reader has to assume it did not happen.

1.3 Standards and frameworks you will use

Reference	What it gives you	Where it appears in the labs
NIST SP 800-86, Guide to Integrating Forensic Techniques into Incident Response (2006)	Four-phase process: collection, examination, analysis, reporting	Lab structure, report outline
NIST SP 800-61 Rev. 3 (April 2025)	Incident response aligned to CSF 2.0; replaced Rev. 2	Context for Lab 3 live response
NIST SP 800-101 Rev. 1	Mobile device forensics guidance	Lab 12
ISO/IEC 27037	Identification, collection, acquisition and preservation of digital evidence	Labs 1, 3, 4

Reference	What it gives you	Where it appears in the labs
ISO/IEC 27041, 27042, 27043	Method suitability, analysis and interpretation, investigation process	Labs 5, 9, capstone
ISO/IEC 17025	Competence of testing and calibration laboratories	Tool validation in Labs 2 and 4
SWGDE best practices	Discipline-specific procedures, validation and reporting guidance	Throughout
Federal Rules of Evidence (Rules 702, 901, 902) and the Daubert standard	Expert testimony, authentication and scientific reliability in U.S. courts	Capstone report and mock testimony
ACPO Good Practice Guide principles	Historically influential UK principles; current national guidance takes precedence	Lab 1 discussion
MITRE ATT&CK	Maps artifacts to adversary tactics and techniques	Labs 6, 11, capstone

1.4 Locard's Exchange Principle, digitally

Edmond Locard's idea is that every contact leaves a trace. On computers this is a useful starting assumption but not a law. Traces may never be written, may be overwritten within minutes, or may be planted or wiped on purpose. In Lab 10 you plant one of those manipulations yourself and then find it.

Part 2: Lab architecture and hardware

The lab runs as three virtual machines and one physical USB flash drive on a single host computer. Two machines are forensic workstations, one Windows and one Linux. The third is the subject machine, the "seized" computer you investigate. Build the virtual machines with the hypervisor you used in the virtualization lab guide (VMware Workstation Pro or VirtualBox).

2.1 Host computer requirements

Component	Minimum	Recommended	Why it matters
CPU	4 cores with VT-x or AMD-V enabled in BIOS/UEFI	8 cores or more	Autopsy ingest and Plaso both scale with cores
RAM	16 GB	32 GB	Lets you run a workstation and the subject at the same time
Free SSD space	400 GB	1 TB	Each disk image, memory dump and timeline is a full copy
Host OS	Windows 10/11, Linux or macOS on Intel	Windows 11 or Fedora/Ubuntu	Apple Silicon Macs cannot run the x64 Windows subject used here
USB port	One USB 3.0 port	Same	For the TRANSFER flash drive

2.2 The virtual machines

Name	Role	Operating system	vCPU	RAM	Disk	Time zone
DF-WIN	Windows forensic workstation	Windows 11 Enterprise evaluation (90 days)	4	16 GB (8 GB minimum)	250 GB	UTC
DF-LNX	Linux forensic workstation	Ubuntu 24.04 LTS Desktop	4	16 GB (8 GB minimum)	250 GB	UTC
HPA-WS07	Subject machine, a workstation at the fictional Harbor Point Analytics	Windows 11 Enterprise evaluation (90 days)	2	4 GB	64 GB	America/Chicago

The subject machine gets only 4 GB of RAM on purpose. Memory dumps are the size of physical memory, so a small subject keeps Lab 3 and Lab 7 fast.

The two workstations run in UTC and the subject runs in Central Time. That mismatch is deliberate. Real investigations always involve more than one time zone, and Lab 9 makes you reconcile them.

2.3 Physical items

Item	Specification	Evidence ID
USB flash drive labelled TRANSFER	8 to 16 GB, wiped and formatted exFAT in Part 5	DF-2026-001-EV01
Optional hardware write blocker	Any USB 3.0 forensic write blocker, such as a Tableau T8u or WiebeTech USB WriteBlocker	Not evidence

The course does not require a hardware write blocker. Lab 4 teaches software write protection and then tests whether it held. If your program owns hardware blockers, use them and record the model and firmware.

2.4 Networks

Network	Type	Address range	Used for
DF-LAB	Host-only	192.168.56.0/24	All lab work. DF-WIN = .10, DF-LNX = .20, HPA-WS07 = .50
NAT	NAT to the internet	Hypervisor default	Installing tools, and the 30-minute seeding window in Part 5 only

Switch every machine back to DF-LAB as soon as installation or seeding is done. A forensic workstation that browses the internet changes its own state and risks contaminating cases.

2.5 Case folder layout

Every case uses the same folder tree on both workstations. On DF-WIN it lives at `C:\Cases`, on DF-LNX at `/cases`.

```
DF-2026-001\
  00_Admin\        authorization, scope, custody forms
  01_Evidence\     original acquisitions, read-only after hashing
  02_Working\      verified working copies you actually analyze
  03_Exports\      tool output: CSV, reports, carved files
  04_Notes\        case notes, one file per examiner
  05_Report\       drafts, exhibits, final report
```

2.6 Naming conventions

- **Case number:** `DF-2026-001` for the shared scenario. Instructors may assign each student a suffix, for example `DF-2026-001-JM`.
- **Evidence items:** `DF-2026-001-EV01`, `EV02` and so on. The `EV` prefix avoids confusion with the `.E01` forensic image format.
- **Files:** `<evidence id>_<description>_<YYYYMMDD>`, for example `DF-2026-001-EV03_HPA-WS07_disk_20261103.E01`.
- **Times in notes:** always UTC, written as `2026-11-03 18:42:10 UTC`. If you record a local time, label the zone.

Part 3: Software inventory

Every tool in this course is free. A few require a short registration form before download. Versions below are the ones current in late October 2026; newer point releases are fine, but record the exact version you install in your case notes, because Principle 6 (Documentation) and Principle 3 (Forensic Soundness) both depend on it.

3.1 Forensic tools

Tool	Version (Oct 2026)	What it does in this course	Installs on	Cost and access	Labs
FTK Imager	4.7.x (free edition)	Disk imaging, image verification, image mounting and preview	DF-WIN	Free, registration form	2, 4

Tool	Version (Oct 2026)	What it does in this course	Installs on	Cost and access	Labs
Autopsy	4.23.x	Graphical case analysis: file systems, deleted files, keyword search, carving	DF-WIN	Free, open source	5, capstone
The Sleuth Kit	4.12 or later	Command line file system analysis (mmls, fls, icat, istat)	DF-LNX	Free, open source	5
Guymager	0.8.x	Graphical forensic imager for Linux, writes E01	DF-LNX	Free, Ubuntu repository	4
dc3dd and ewf-tools	Ubuntu 24.04 packages	Command line imaging with hashing; E01 verification and info	DF-LNX	Free, Ubuntu repository	2, 4
WinPmem	4.0 RC2 (winpmem_mini_x64)	Physical memory acquisition on a live Windows system	Response USB, run on HPA-WS07	Free, open source	3
Volatility 3	2.28 or later	Memory analysis: processes, command lines, network, injected code	DF-LNX	Free, open source	7
Wireshark	4.6.x	Packet capture analysis	DF-WIN and DF-LNX	Free, open source	8
Zeek	8.0 LTS	Turns packet captures into connection, DNS, HTTP and file logs	DF-LNX	Free, open source	8
KAPE	1.3.x	Targeted Windows artifact collection and processing	DF-WIN	Free for this use, registration form	6

Tool	Version (Oct 2026)	What it does in this course	Installs on	Cost and access	Labs
Eric Zimmerman's Tools	Current .NET 9 builds	Parsers for MFT, registry, event logs, Prefetch, LNK, Jump Lists, Amcache, Shimcache, Recycle Bin; Timeline Explorer and Registry Explorer	DF-WIN	Free	6, 9, 10
Plaso (log2timeline)	Current Docker image	Super timeline generation across hundreds of artifact types	DF-LNX (Docker)	Free, open source	9
Velociraptor	0.77.3 or later	Endpoint collection and hunting with VQL artifacts	DF-LNX (server), HPA-WS07 clone (client)	Free, open source	11
ALEAPP and iLEAPP	Current release	Android and iOS artifact parsing into HTML reports	DF-WIN	Free, open source	12
ExifTool	13.x	Read and write file metadata	DF-WIN, DF-LNX, HPA-WS07 (seeding only)	Free, open source	10
YARA	4.5.x	Pattern-based file identification	DF-WIN and DF-LNX	Free, open source	11
Ghidra	12.1.x	Reverse engineering and static program analysis	DF-WIN	Free, open source, needs JDK 21	11

The Velociraptor project asks all users to run 0.77.3 or later to close several recent CVEs, including CVE-2026-5329. Do not install an older build from a mirror.

3.2 Supporting software

Software	Purpose	Installs on
7-Zip	Extracting tool archives and datasets	DF-WIN

Software	Purpose	Installs on
Python 3.12 (64-bit)	Runs ALEAPP, iLEAPP and optional Volatility on Windows	DF-WIN
.NET Desktop Runtime 9	Required by the Eric Zimmerman .NET 9 builds	DF-WIN
Eclipse Temurin JDK 21 (64-bit)	Required by Ghidra	DF-WIN
Docker Engine	Runs the Plaso container	DF-LNX
pipx	Isolated install of Volatility 3	DF-LNX
DB Browser for SQLite	Manual verification of SQLite databases parsed by tools	DF-WIN
Packet Monitor (pktmon)	Built into Windows; acts as the company network sensor on HPA-WS07	HPA-WS07

3.3 Built-in commands you will use constantly

Task	Windows (PowerShell)	Linux
Hash a file	`Get-FileHash -Algorithm SHA256 <file>`	`sha256sum <file>`
Current UTC time for notes	`(Get-Date).ToUniversalTime().ToString('u')`	`date -u '+%F %T UTC'`
List disks	`Get-Disk`	`lsblk -o NAME,SIZE,MODEL,SERIAL,RO`
Make a device read-only	Registry WriteProtect policy (Lab 4)	`sudo blockdev --setro /dev/sdX`

Part 4: Download, install and configure the software

This part takes about six hours spread over two sessions. Work through it in order: build the virtual machines (4.1), harden them for forensic use (4.2), install the Windows tools (4.3), install the Linux tools (4.4), build the response kit (4.5), then run the smoke tests

(4.6). Every install ends with a check that proves it worked. Do not skip the checks; a tool that half-installed will fail in the middle of a lab, usually at the worst moment.

4.0 Rules for installing forensic software

1. **Download only from the publisher.** Use the links in Part 3. Mirror sites and "download portals" bundle adware and sometimes alter the binary.
2. **Verify every download.** If the publisher posts a SHA-256 hash, compare it. On Windows: `Get-FileHash .\file.zip -Algorithm SHA256`. On Linux: `sha256sum file.zip`. A mismatch means delete and download again.
3. **Record what you installed.** Fill in one line of the Tool Validation Log (Appendix F) per tool: name, version, source URL, SHA-256 of the installer, date installed.
4. **Install on NAT, work on DF-LAB.** Attach the NAT adapter for installation, then disconnect it before any lab work.
5. **Snapshot when finished.** Take a snapshot named `Clean-Tools-<date>` on each workstation. If a tool or a lab goes badly, you roll back to a known state instead of guessing what changed.

4.1 Build the virtual machines

You built Windows and Ubuntu virtual machines in the virtualization lab guide. Use the same process here with the settings from Part 2.

Download and verify the operating system images

1. **Windows 11 Enterprise evaluation.** Go to the Microsoft Evaluation Center, choose *Download the ISO*, complete the registration form, and pick *ISO, Enterprise download, 64-bit, English (United States)*. The evaluation runs 90 days and needs no product key.
2. Hash the ISO and compare it with the SHA-256 listed on the Evaluation Center download page:

   ```
   Get-FileHash .\<windows-iso-name>.iso -Algorithm SHA256
   ```

3. **Ubuntu 24.04 LTS Desktop.** Download the ISO from ubuntu.com/download/desktop. Download `SHA256SUMS` from the same release folder and run:

   ```
   sha256sum -c SHA256SUMS 2>/dev/null | grep OK
   ```

   The line for your ISO must read `OK`.

Create DF-WIN

1. New virtual machine, Windows 11 x64, 4 vCPU, 16 GB RAM, 250 GB disk (thin provisioned or dynamically allocated is fine).
2. Windows 11 requires TPM 2.0 and Secure Boot. In VMware Workstation, enable *Encryption* and add a *Trusted Platform Module*. In VirtualBox 7, choosing the Windows 11 template enables TPM 2.0 and Secure Boot automatically.
3. Network adapter 1: NAT. Network adapter 2: host-only network DF-LAB.
4. Install Windows. At the account screen choose *Sign-in options, Domain join instead* to create a local account named `examiner`. Forensic workstations should never sign in with a personal Microsoft account.
5. Install VMware Tools or VirtualBox Guest Additions so the display and clipboard work.

Create DF-LNX

1. New virtual machine, Ubuntu 64-bit, 4 vCPU, 16 GB RAM, 250 GB disk.
2. Network adapter 1: NAT. Network adapter 2: DF-LAB.
3. Install Ubuntu with the *Normal installation* option. Create the user `examiner`.
4. After first boot, update and add guest tools:

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y open-vm-tools-desktop   # VMware only; VirtualBox
users install Guest Additions from the Devices menu
sudo reboot
```

4.2 Harden the workstations for forensic use

A normal desktop is built to be helpful: it mounts drives the moment you plug them in, syncs the clock, and opens files automatically. A forensic workstation must be built to do nothing on its own.

DF-WIN hardening

Open PowerShell as Administrator and run each block.

1. Set the clock to UTC and name the machine:

```
Set-TimeZone -Id "UTC"
Rename-Computer -NewName "DF-WIN" -Restart
```

2. Stop Windows from automatically mounting new volumes. This is your first line of defence against writing to evidence, because Windows writes metadata to any NTFS volume it mounts:

   ```
   mountvol /N
   ```

   Check it with `diskpart`, then type `automount`. It should report *Automatic mounting of new volumes disabled*.
3. Turn off AutoPlay: *Settings, Bluetooth & devices, AutoPlay*, set *Use AutoPlay for all media and devices* to Off.
4. Show file extensions and hidden files:

   ```
   $k = 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Explorer\Advanced'
   Set-ItemProperty $k HideFileExt 0
   Set-ItemProperty $k Hidden 1
   Set-ItemProperty $k ShowSuperHidden 1
   Stop-Process -Name explorer
   ```

5. Create the tool and case folders and stop Microsoft Defender from quarantining your tools and evidence. Defender will otherwise delete YARA rules, the Velociraptor binary and anything in a case that looks malicious, which destroys evidence. This exclusion is acceptable only because DF-WIN never browses the internet during casework:

   ```
   New-Item -ItemType Directory C:\Tools, C:\Cases | Out-Null
   Add-MpPreference -ExclusionPath 'C:\Tools','C:\Cases'
   ```

6. Give the DF-LAB adapter its static address. Find its name with `Get-NetAdapter` (usually `Ethernet1`):

   ```
   New-NetIPAddress -InterfaceAlias "Ethernet1" -IPAddress 192.168.56.10 -
   PrefixLength 24
   ```

7. Share the case folder so DF-LNX can read images from it:

   ```
   New-SmbShare -Name Cases -Path C:\Cases -FullAccess "DF-WIN\examiner"
   ```

DF-LNX hardening

1. Set UTC, name the host and create the case root:

```
sudo timedatectl set-timezone UTC
sudo hostnamectl set-hostname DF-LNX
sudo mkdir -p /cases /opt/tools && sudo chown examiner:examiner /cases
/opt/tools
```

2. Stop GNOME from mounting and opening removable media:

```
gsettings set org.gnome.desktop.media-handling automount false
gsettings set org.gnome.desktop.media-handling automount-open false
gsettings set org.gnome.desktop.media-handling autorun-never true
```

3. Give the DF-LAB adapter its static address. List connections with `nmcli con show`; the DF-LAB one is usually `Wired connection 2`:

```
sudo nmcli con mod "Wired connection 2" ipv4.method manual ipv4.addresses
192.168.56.20/24
sudo nmcli con up "Wired connection 2"
ping -c 2 192.168.56.10
```

   Two replies from DF-WIN means the lab network works. If Windows Firewall blocks ping, run `Enable-NetFirewallRule -Name FPS-ICMP4-ERQ-In` on DF-WIN.

4.3 Install the Windows tools on DF-WIN

Connect DF-WIN's NAT adapter. Open PowerShell as Administrator.

Step 1: Supporting software through winget

Windows Package Manager (winget) ships with Windows 11 and pulls installers directly from the publishers.

```
winget install -e --id 7zip.7zip
winget install -e --id Python.Python.3.12
winget install -e --id Microsoft.DotNet.DesktopRuntime.9
winget install -e --id EclipseAdoptium.Temurin.21.JDK
winget install -e --id DBBrowserForSQLite.DBBrowserForSQLite
winget install -e --id WiresharkFoundation.Wireshark
```

Close and reopen PowerShell so the new PATH entries load, then check:

```
py --version          # Python 3.12.x
dotnet --list-runtimes # Microsoft.WindowsDesktop.App 9.x listed
java -version         # openjdk version "21.x"
```

The Wireshark installer offers Npcap. DF-WIN only analyzes captures, so Npcap is optional; leave it unticked if you want the workstation unable to sniff.

Step 2: FTK Imager

1. Open the FTK Imager page and choose the free download. Complete the form; Exterro emails the download link.
2. Download the 4.7.x installer to `C:\Tools\_installers`, hash it, and record the hash.
3. Run the installer and accept the defaults.
4. Launch FTK Imager with *Run as administrator*. Physical drives only appear when it runs elevated.
5. **Check:** *Help, About* shows the version. *File, Add Evidence Item, Physical Drive* lists `\\.\PHYSICALDRIVE0`.

Step 3: Autopsy

1. Download the 64-bit MSI from autopsy.com/download. Autopsy releases are also on the Sleuth Kit GitHub releases page, which lists SHA-256 hashes.
2. Run the MSI and accept the defaults.
3. Launch Autopsy, close the Welcome window, and open *Tools, Options*:
   - *Application* tab: set *Maximum JVM memory* to 8 GB (half of DF-WIN's RAM). Restart Autopsy when asked.
   - *Central Repository* tab: tick *Use a Central Repository* and keep the default SQLite database. This lets Autopsy correlate files across your cases later.
4. **Check:** *Help, About* shows Autopsy 4.23.x and The Sleuth Kit version.

Step 4: Eric Zimmerman's Tools

1. Download Get-ZimmermanTools.zip and extract it to `C:\Tools\EZTools`.
2. Run the downloader for the .NET 9 builds:

```
Set-Location C:\Tools\EZTools
Unblock-File .\Get-ZimmermanTools.ps1
.\Get-ZimmermanTools.ps1 -Dest C:\Tools\EZTools -NetVersion 9
```

3. Add the folder to the system PATH so the command line tools run from anywhere:

```
[Environment]::SetEnvironmentVariable('Path', $env:Path +
';C:\Tools\EZTools', 'Machine')
```

4. **Check:** open a new PowerShell window and run `MFTECmd.exe`. It prints its version and usage. Then open `C:\Tools\EZTools\TimelineExplorer\TimelineExplorer.exe` and `C:\Tools\EZTools\RegistryExplorer\RegistryExplorer.exe`; both should start without a .NET error.

Step 5: KAPE

1. Request KAPE through the form at kroll.com/kape. Turn off ad blockers or use a private window, or the form may fail silently. The download link arrives by email.
2. Extract the zip to `C:\Tools\KAPE` and unblock the files:

```
Get-ChildItem C:\Tools\KAPE -Recurse | Unblock-File
```

3. Update the Targets and Modules from the community repository:

```
Set-Location C:\Tools\KAPE
.\kape.exe --sync
```

4. KAPE Modules call the Zimmerman tools from `C:\Tools\KAPE\Modules\bin`. Populate that folder with the community updater script. Download `KAPE-EZToolsAncillaryUpdater.ps1` from github.com/AndrewRathbun/KAPE-EZToolsAncillaryUpdater into `C:\Tools\KAPE`, then run it from that folder and choose the .NET 9 builds when it asks.
5. **Check:** start `gkape.exe` as Administrator. Tick *Use Target options* and confirm `KapeTriage` appears. Tick *Use Module options* and confirm `!EZParser` appears. Confirm the file `C:\Tools\KAPE\Modules\bin\EvtxECmd\EvtxECmd.exe` exists.

KAPE is free for this coursework. Kroll requires an enterprise license when KAPE is used on a third-party network or as part of a paid engagement, so do not carry this copy into consulting work.

Step 6: ExifTool

1. Download the Windows 64-bit zip from exiftool.org and check its SHA-256 against the checksum list on the same page.

2. Extract it to `C:\Tools\ExifTool`. Rename `exiftool(-k).exe` to `exiftool.exe` and keep the `exiftool_files` folder beside it.
3. Add `C:\Tools\ExifTool` to the system PATH the same way as Step 4.
4. **Check:** `exiftool -ver` prints 13.xx.

Step 7: YARA

1. From the YARA releases page, download the latest `yara-v4.5.x-...-win64.zip`.
2. Extract to `C:\Tools\YARA` and add that folder to PATH.
3. **Check:** `yara64.exe --version` prints 4.5.x.

Step 8: Ghidra

1. From Ghidra releases, download the asset named `ghidra_12.1.x_PUBLIC_<date>.zip`. Do not download the *Source code* archives. Compare the SHA-256 shown on the release page.
2. Extract to `C:\Tools\` so the program lives in `C:\Tools\ghidra_12.1.x_PUBLIC`. Never extract a new version over an old one.
3. Run `ghidraRun.bat`. If Ghidra asks for a JDK, point it to `C:\Program Files\Eclipse Adoptium\jdk-21...`, the JDK folder itself, not a JRE.
4. **Check:** the Ghidra project window opens. *File, New Project, Non-Shared Project*, name it `DF-2026-001`, save it under `C:\Cases\DF-2026-001\02_Working\ghidra`.

Step 9: ALEAPP and iLEAPP

The release builds need no Python setup.

1. From ALEAPP releases and iLEAPP releases, download the latest Windows GUI zips.
2. Extract to `C:\Tools\ALEAPP` and `C:\Tools\iLEAPP`.
3. **Check:** `aleappGUI.exe` and `ileappGUI.exe` both open their window.

If a release build will not start, use Python instead: download the source zip, extract it, and run `py -m pip install -r requirements.txt` followed by `py aleappGUI.py` from the extracted folder.

Step 10: Finish DF-WIN

1. Disconnect the NAT adapter.
2. Fill in the Tool Validation Log (Appendix F) for every tool above.
3. Shut down and take the snapshot `Clean-Tools-<date>`.

4.4 Install the Linux tools on DF-LNX

Connect DF-LNX's NAT adapter and open a terminal.

Step 1: Packages from the Ubuntu repositories

```
sudo apt update
sudo apt install -y sleuthkit dc3dd guymager ewf-tools wireshark tshark yara
\
  libimage-exiftool-perl pipx docker.io genisoimage cifs-utils jq git curl
unzip python3-venv
```

When the Wireshark installer asks *Should non-superusers be able to capture packets?*, answer **No**. DF-LNX analyzes captures; it does not take them.

Check each tool reports a version:

```
fls -V            # The Sleuth Kit ver 4.12.x or later
dc3dd --version
ewfinfo -V
tshark -v | head -1
yara --version
exiftool -ver
sudo guymager &   # the Guymager window opens and lists the attached disks
```

Step 2: Volatility 3

1. Install it in its own isolated environment with pipx. The `[full]` extra pulls in the optional libraries that the malware and YARA plugins need:

   ```
   pipx ensurepath
   pipx install "volatility3[full]"
   ```

2. Close and reopen the terminal, then run `vol -h | head -5`. You should see the Volatility 3 Framework banner and version 2.28 or later.
3. Download the Windows symbol pack so analysis works offline:

   ```
   mkdir -p /opt/tools/vol3-symbols && cd /opt/tools/vol3-symbols
   wget
   https://downloads.volatilityfoundation.org/volatility3/symbols/windows.zip
   wget
   https://downloads.volatilityfoundation.org/volatility3/symbols/SHA256SUMS
   sha256sum -c SHA256SUMS --ignore-missing   # windows.zip: OK
   ```

4. You will always point Volatility at this folder with `-s /opt/tools/vol3-symbols`. The first run against a new memory image builds a symbol cache and can take several minutes.

A symbol pack only covers Windows builds that existed when it was made. If Lab 7 reports that no symbol table was found, the subject runs a newer build. Reconnect NAT for that one command so Volatility can fetch the matching symbols from Microsoft, then disconnect again and record it in your notes.

Step 3: Zeek 8.0 LTS

Zeek is not in the Ubuntu repositories. The Zeek project publishes packages through the openSUSE Build Service.

```
echo 'deb
https://download.opensuse.org/repositories/security:/zeek/xUbuntu_24.04/ /' \
  | sudo tee /etc/apt/sources.list.d/security:zeek.list
curl -fsSL
https://download.opensuse.org/repositories/security:zeek/xUbuntu_24.04/Releas
e.key \
  | gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/security_zeek.gpg >
/dev/null
sudo apt update
sudo apt install -y zeek-8.0
echo 'export PATH=$PATH:/opt/zeek/bin' >> ~/.bashrc && source ~/.bashrc
```

If a Postfix mail configuration screen appears, choose *No configuration*.

Check: `zeek --version` prints 8.0.x, and `which zeek-cut` returns `/opt/zeek/bin/zeek-cut`.

Step 4: Docker and Plaso

The Plaso team recommends the Docker image as the most reliable way to run current Plaso.

1. Let your account run Docker, then log out and back in:

   ```
   sudo usermod -aG docker $USER
   ```

2. Pull and test the image:

```
docker pull log2timeline/plaso
docker run --rm log2timeline/plaso log2timeline.py --version
docker images --digests log2timeline/plaso
```

3. Copy the image digest (the `sha256:...` value) into the Tool Validation Log. The digest, not the tag, identifies exactly which Plaso build you used, which is what Principle 5 (Reproducibility) needs.
4. Add a shortcut that mounts the case folder into the container:

```
echo "alias plaso='docker run --rm -v /cases:/cases log2timeline/plaso'"
>> ~/.bashrc
source ~/.bashrc
```

Step 5: Velociraptor

1. Open the Velociraptor downloads page and download two release assets for version 0.77.3 or later: the `linux-amd64` binary and the `windows-amd64.exe` binary. Save both in `/opt/tools/velociraptor`.
2. Make the Linux binary executable and put it on the PATH (adjust the file name to your version):

```
cd /opt/tools/velociraptor
chmod +x velociraptor-v0.77.3-linux-amd64
sudo ln -s /opt/tools/velociraptor/velociraptor-v0.77.3-linux-amd64
/usr/local/bin/velociraptor
```

3. **Check:** `velociraptor version` prints the version and build time. Server configuration happens in Lab 11.

Step 6: Read-only access to DF-WIN's case share

DF-LNX reads images that DF-WIN acquired. Mount the share read-only so nothing on DF-LNX can change them:

```
sudo mkdir -p /mnt/dfwin
sudo mount -t cifs //192.168.56.10/Cases /mnt/dfwin \
  -o ro,username=examiner,vers=3.0,uid=$(id -u),gid=$(id -g)
touch /mnt/dfwin/test.txt   # must fail with "Read-only file system"
```

That failing `touch` is the check. If it succeeds, unmount, delete the test file on DF-WIN, and repeat with `ro`.

Step 7: Finish DF-LNX

Disconnect NAT, complete the Tool Validation Log, shut down, and take the snapshot `Clean-Tools-<date>`.

4.5 Build the response kit

When you respond to a live machine you bring your own trusted tools on read-only media and write output to separate media. You never run tools from the subject's disk or save output to it. In this lab the read-only media is an ISO file mounted as a virtual CD, and the output media is a second virtual disk.

Build the read-only toolkit ISO on DF-LNX

1. With NAT connected, download `winpmem_mini_x64_rc2.exe` from the WinPmem releases page into a new folder:

   ```
   mkdir -p /opt/tools/response && cd /opt/tools/response
   # save winpmem_mini_x64_rc2.exe here from the releases page
   ```

2. Create the live response script `live_response.cmd` in the same folder with the contents printed in Lab 3, step 6.
3. Write a manifest of hashes, then build and hash the ISO:

   ```
   sha256sum * > KIT_MANIFEST.sha256
   genisoimage -o /cases/DF-RESPONSE-KIT.iso -J -R -V DFRESPONSE
   /opt/tools/response
   sha256sum /cases/DF-RESPONSE-KIT.iso
   ```

4. Move the ISO to the host through a temporary hypervisor shared folder, then remove the shared folder. Record the ISO hash in the Tool Validation Log. Because an ISO cannot be written to, the subject machine cannot alter your tools, and anyone can later prove which tool versions you ran.

Create the RESPONSE-OUT disk

1. In the hypervisor, create a new 16 GB virtual disk named `RESPONSE-OUT` and attach it to DF-WIN.
2. On DF-WIN open *Disk Management* (`diskmgmt.msc`). Initialize the new disk as GPT, create one NTFS volume labelled `RESPONSE-OUT`, and assign it a drive letter. Automount is off, so you must assign the letter yourself.

3. Create the folder `DF-2026-001` on it, then shut down DF-WIN and detach the disk. It stays detached until Lab 3.

4.6 Smoke tests

Run every row before starting Part 5. Write the actual output in your Tool Validation Log.

Workstation	Tool	Command or action	Pass when
DF-WIN	FTK Imager	Run as admin, *Add Evidence Item, Physical Drive*	PHYSICALDRIVE0 is listed
DF-WIN	Autopsy	*Help, About*	4.23.x shown
DF-WIN	EZ Tools	`MFTECmd.exe`	Usage and version print
DF-WIN	KAPE	`kape.exe --tlist .`	Target list prints, includes KapeTriage
DF-WIN	ExifTool	`exiftool -ver`	13.xx
DF-WIN	YARA	`yara64.exe --version`	4.5.x
DF-WIN	Ghidra	`ghidraRun.bat`	Project window opens
DF-WIN	ALEAPP / iLEAPP	Launch GUI	Window opens
DF-LNX	Sleuth Kit	`fls -V`	Version prints
DF-LNX	dc3dd / ewf-tools	`dc3dd --version`, `ewfinfo -V`	Versions print
DF-LNX	Volatility 3	`vol -h`	Version 2.28 or later
DF-LNX	Zeek	`zeek --version`	8.0.x
DF-LNX	Plaso	`plaso log2timeline.py --version`	Version and date print
DF-LNX	Velociraptor	`velociraptor version`	0.77.3 or later
DF-LNX	Share	`touch /mnt/dfwin/x`	Fails, read-only

Part 5: Build the evidence scenario

This is the step that makes the whole course work. You play out a small insider data theft on the subject machine HPA-WS07. Because you perform every action yourself, you know the ground truth, so every finding in Labs 1 to 12 can be checked against what actually happened. The scenario is fictional and runs entirely inside your lab.

5.1 The case, in brief

> Harbor Point Analytics suspects that an employee, Dana Reyes, copied a confidential client pricing file to a personal USB drive and emailed a second copy to a private address shortly before resigning. IT imaged her workstation (HPA-WS07) and preserved a USB drive found at her desk. You are the examiner.

What you will actually make the machine do, and the artifacts each action leaves behind:

Seeding action	Artifacts it creates	Lab that finds it
Log in as `dreyes`, work normally for a few minutes	Registry user profile, event log sign-in (EID 4624), Prefetch	6
Open the file `Q4_Pricing_CONFIDENTIAL.xlsx` from a network folder	LNK file, Jump List, RecentDocs, Office MRU	6, 9
Rename it to `lunch_menu.xlsx` to disguise it	$MFT entry, $UsnJrnl change record	6, 10
Change the file's timestamps with a tool (timestomping)	$MFT $STANDARD_INFORMATION vs $FILE_NAME mismatch	10
Copy it to the TRANSFER USB drive	Registry USBSTOR, setupapi log, LNK on USB, file on USB	6
Email the file to a personal address through webmail	Browser history, cache, a packet capture of the upload	7, 8
Download and run a small "cleaner" utility	Amcache, Prefetch, Defender/AV log, a YARA-matchable file	6, 11
Delete the original and clear one event log	Recycle Bin `$I` / `$R`, EID 1102 log-cleared record	6, 9

Seeding action	Artifacts it creates	Lab that finds it
Leave the machine running	Live memory with the browser and cleaner still in RAM	3, 7

None of these actions are harmful. The "cleaner" is a harmless script you write yourself. No real malware is used anywhere in this course.

5.2 Prepare to seed

1. Boot HPA-WS07 with the NAT adapter connected so you can reach webmail. Set its clock to Central Time and confirm the date is correct; the timeline labs depend on it.
2. Create a standard (non-admin) local user `dreyes`, and a separate admin user `itadmin` you will use for setup only.
3. On DF-LNX, create the "confidential" file and a fake network share, or simply place the file in a folder on HPA-WS07 named `P:\ClientData` to stand in for a mapped drive. The file is an ordinary spreadsheet you create in Excel or LibreOffice titled `Q4_Pricing_CONFIDENTIAL.xlsx` with a few rows of invented prices.
4. Insert the TRANSFER USB drive. As `itadmin`, wipe and format it exFAT, volume label `TRANSFER`. This is evidence item `DF-2026-001-EV01`.
5. Write the harmless cleaner. Create `C:\Users\Public\cleanup.ps1` containing a script that only prints messages and writes a marker file, plus a unique string you will hunt for later, for example `HPA-CLEANER-SIGNATURE-7731`. Nothing in it deletes or encrypts anything.

5.3 Seed the activity

Do all of this signed in as `dreyes` unless noted. Keep the NAT window under 30 minutes. Note the real UTC time as you start; you will compare it later.

1. Sign in as `dreyes`. Open a browser, read a news site for a minute, open the calculator, open Notepad. This creates normal background activity so the theft does not stand alone.
2. Open `P:\ClientData\Q4_Pricing_CONFIDENTIAL.xlsx`, scroll through it, and close it.
3. Copy the file to the desktop. Rename the desktop copy to `lunch_menu.xlsx`.
4. Timestomp the renamed copy. In an elevated PowerShell (as `itadmin`), set its timestamps back three months:

```
$f = "C:\Users\dreyes\Desktop\lunch_menu.xlsx"
$t = (Get-Date).AddMonths(-3)
(Get-Item $f).CreationTime = $t
(Get-Item $f).LastWriteTime = $t
(Get-Item $f).LastAccessTime = $t
```

This changes only the values Windows shows ($STANDARD_INFORMATION). The $MFT $FILE_NAME timestamps keep the real time, which is exactly the mismatch Lab 10 detects.

5. Copy `lunch_menu.xlsx` to the `TRANSFER` drive, then safely eject the drive. Set it aside as evidence EV01.
6. Open webmail in the browser, sign in to a throwaway account you control, and email `lunch_menu.xlsx` to another address you control. (The packet capture in step 9 records this; you do not need the message itself.)
7. As `dreyes`, copy `C:\Users\Public\cleanup.ps1` to the desktop and run it once:

```
powershell -ExecutionPolicy Bypass -File
"$env:USERPROFILE\Desktop\cleanup.ps1"
```

8. Delete the original `Q4_Pricing_CONFIDENTIAL.xlsx` from the desktop to the Recycle Bin. Then, as `itadmin`, clear one event log to simulate a cover-up:

```
wevtutil cl "Windows PowerShell"
```

9. The network capture. Before step 6, start the built-in Windows packet monitor as `itadmin` so the webmail upload is recorded. This stands in for the company network sensor:

```
pktmon start --capture --pkt-size 0 --file C:\Users\Public\hpa_capture.etl
# ... perform step 6 ...
pktmon stop
pktmon etl2pcap C:\Users\Public\hpa_capture.etl --out
C:\Users\Public\hpa_capture.pcapng
```

Copy `hpa_capture.pcapng` off the machine later as evidence EV04. It is the packet capture for Labs 8.

5.4 Freeze the scene

1. **Do not shut down.** Leave HPA-WS07 running and signed in. Lab 3 acquires its live memory, and the browser and cleaner must still be resident in RAM.
2. Disconnect the NAT adapter and attach the DF-LAB adapter. The scene is now isolated.
3. In the hypervisor, take a snapshot named `SEIZED-STATE`. This is your master copy of the scene. If any later lab corrupts the machine, revert to this snapshot and you are back to the moment of seizure. Treat `SEIZED-STATE` as the original evidence: hash-protected by the fact that you never modify it, only clone from it.
4. Record in the case notes the real UTC time of seizure and the subject's displayed local time. The gap between them is a finding in Lab 9.

The subject machine is now ready. Everything from here is investigation.

Part 6: The labs

Twelve labs take you through the full investigation of Case DF-2026-001, roughly in the order of the seven pillars. Each one opens an evidence item, pulls out specific artifacts, and checks them against the ground truth you seeded in Part 5. Do them in order; later labs depend on evidence that earlier labs acquire.

Before every working session, clone `SEIZED-STATE` rather than booting it, and open your case notes (Appendix C). Write the current UTC time at the top of each session.

Lab 1: Open the case, scope, authority and chain of custody

Objectives. Define scope, record legal authority, and start a chain of custody that will survive scrutiny.

Principles reinforced. 2 (Chain of Custody), 9 (Legal Authority), 6 (Documentation).

Prerequisites. Part 5 complete. The subject is seized and snapshotted.

Procedure.

1. In `00_Admin`, complete the Authorization Record (Appendix A, top section): who authorized the examination, what it covers, and its limits. For this course the authority is the (fictional) Harbor Point Analytics internal investigation consent signed by its CISO. Write one sentence on what you are NOT authorized to examine, for example personal cloud accounts.

2. Write the scope statement: the question you are answering (did Dana Reyes exfiltrate the pricing file, and how), the evidence in scope (HPA-WS07 and the TRANSFER drive), and the time window of interest.
3. Open a chain of custody form (Appendix A) for each evidence item: EV01 (TRANSFER USB), and the HPA-WS07 image and memory you are about to create. Fill in the first custody line for EV01 now: received from IT, date, your name.
4. Start your case notes file in `04_Notes`. The first entry records the seizure time and local time you noted in Part 5.

Intended result. A complete `00_Admin` folder: authorization, scope, custody forms, and a notes file with its first timestamped entry.

Why this result matters. A defensible investigation is defined before any tool runs. Authority answers "were you allowed to look?" and scope answers "did you stay within what you were allowed to look at?" If either is missing, strong technical findings can be excluded. Starting custody now, not later, is what makes the evidence traceable end to end.

Deliverables. The four `00_Admin` documents.

Check your understanding.

- What is the difference between authority and scope?
- Why does the first custody entry for EV01 matter more than a later one?
- Name one thing outside your scope here and why.

Lab 2: Hashing, verification and the forensic soundness of a tool

Objectives. Understand cryptographic hashing, prove a copy is identical to its source, and validate a tool against a known answer before trusting it.

Principles reinforced. 1 (Evidence Integrity), 3 (Forensic Soundness), 4 (Repeatability).

Prerequisites. Labs 1 done. Work on DF-LNX.

Procedure.

1. Create a small known test file and hash it two ways:

```
echo "Harbor Point Analytics test vector" > /cases/DF-2026-
001/02_Working/testvector.txt
sha256sum /cases/DF-2026-001/02_Working/testvector.txt
md5sum    /cases/DF-2026-001/02_Working/testvector.txt
```

2. Change one character in the file and hash again. Record how much of the hash changed.
3. Make a byte-for-byte copy and confirm the copy's SHA-256 matches the original's exactly.
4. Validate your imaging tool against a reference image. Download a small public test image (for example from the Digital Corpora teaching corpus), note its published hash, image it or copy it with `dc3dd`, and confirm your result hashes to the published value.
5. Record the validation in the Tool Validation Log: tool, version, reference dataset, expected hash, obtained hash, pass or fail.

Intended result. Identical copies produce identical SHA-256 values; a one-character change produces a completely different hash; your imaging tool reproduces a known reference hash.

Why this result matters. Hashing is how integrity is proven in practice. Because a hash changes completely from a single altered bit (the avalanche effect), a matching hash is strong evidence that nothing changed. Validating the tool against a known answer first is Forensic Soundness: you trust the tool because you tested it, not because of its reputation. Note that MD5 is cryptographically broken and is kept only for cross-checking against older records; SHA-256 is your primary hash.

Deliverables. The hash comparison notes and a completed Tool Validation Log row.

Check your understanding.

- Why does one changed character change most of the hash?
- Why do we still sometimes record MD5?
- What would a hash mismatch on your copy tell you?

Lab 3: Live response and memory acquisition, order of volatility

Objectives. Acquire volatile evidence from the running subject in the right order, using trusted tools on read-only media, while leaving the smallest possible footprint.

Principles reinforced. 7 (Order of Volatility), 8 (Least Intrusion), 1 (Integrity), 2 (Chain of Custody).

Prerequisites. `SEIZED-STATE` snapshot, the response kit ISO and the RESPONSE-OUT disk from Part 4.5.

Procedure.

1. Revert HPA-WS07 to `SEIZED-STATE` so it is running exactly as seized. Keep it on DF-LAB only.
2. Attach the `DF-RESPONSE-KIT.iso` as a virtual CD and the `RESPONSE-OUT` virtual disk. Note the exact UTC time you begin; this is the start of your footprint.
3. Open the response script `live_response.cmd` from the CD (never from C:). It should capture, in volatility order, to the RESPONSE-OUT disk:

   ```
   @echo off
   set OUT=E:\DF-2026-001
   set KIT=D:\
   echo Start %DATE% %TIME% > %OUT%\response_log.txt
   %KIT%\winpmem_mini_x64_rc2.exe %OUT%\HPA-WS07_mem.raw >>
   %OUT%\response_log.txt 2>&1
   date /t >> %OUT%\response_log.txt & time /t >> %OUT%\response_log.txt
   ```

   (Adjust `D:` and `E:` to the CD and output drive letters on your subject.)
4. From the same CD, record quick volatile context to the output disk: `ipconfig /all`, `netstat -ano`, `arp -a`, `tasklist`. Redirect each to a file on RESPONSE-OUT. These are the fastest-changing items and are captured right after memory.
5. Hash the memory image immediately:

   ```
   certutil -hashfile %OUT%\HPA-WS07_mem.raw SHA256 >> %OUT%\response_log.txt
   ```

6. Note the exact UTC time you finish. Shut down HPA-WS07. Move the memory image and logs to `01_Evidence` as `DF-2026-001-EV02`, and add a custody line.

Intended result. A raw memory image of a few GB, a SHA-256 hash recorded at acquisition, and small text captures of network and process state. The response log shows a start and end time only a few minutes apart.

Why this result matters. Order of Volatility says capture what disappears first: RAM vanishes on shutdown, network connections change by the second, the disk survives. You captured memory before touching the disk. Running tools from read-only media and writing only to separate media is Least Intrusion; the few megabytes your tools wrote to RAM are the unavoidable footprint, which is why you recorded exactly when you were on the machine. Compare that footprint against doing nothing: you accept a small, documented change in order to save evidence that would otherwise be lost.

Deliverables. EV02 memory image with its hash, the response log, the volatile text captures, and a custody line.

Check your understanding.

- Why capture memory before imaging the disk?
- Why run the tools from a CD image rather than the subject's own copy of them?
- What footprint did you leave, and how did you document it?

Lab 4: Disk imaging and write protection

Objectives. Create a verified forensic image of the subject's disk, and prove your write protection held.

Principles reinforced. 2 (Chain of Custody), 1 (Integrity), 3 (Forensic Soundness), 4 (Repeatability).

Prerequisites. HPA-WS07 shut down after Lab 3.

Procedure.

1. Export HPA-WS07's virtual disk to a raw file to stand in for a physically removed drive: in the hypervisor, clone or convert its `.vmdk` / `.vdi` to a raw `.dd` (for example `qemu-img convert -O raw HPA-WS07.vmdk hpa-ws07-raw.dd`). Place it where DF-WIN can read it.
2. **Image on DF-WIN with FTK Imager.** Run FTK Imager as admin. *File, Create Disk Image, Image File*, select the raw source, and choose the **E01** format. Enter case metadata (case number DF-2026-001, examiner, notes). Tick *Verify images after they are created*. Write the output to `01_Evidence` as `DF-2026-001-EV03_HPA-WS07_disk_<date>.E01`.
3. When imaging finishes, FTK Imager shows a verification result with source and image MD5 and SHA-1. They must match. Save the generated `.txt` summary.
4. **Reproduce on DF-LNX.** Independently image the same source with `dc3dd`, hashing as you go, then confirm the E01 with `ewfverify`:

   ```
   dc3dd if=/mnt/source/hpa-ws07-raw.dd of=/cases/DF-2026-
   001/02_Working/hpa_lnx.raw hash=sha256 log=/cases/DF-2026-
   001/02_Working/dc3dd.log
   ewfverify /mnt/dfwin/DF-2026-001/01_Evidence/DF-2026-001-EV03_HPA-
   WS07_disk_<date>.E01
   ```

5. **Test write protection.** On DF-LNX, set the source read-only and prove a write fails:

```
sudo blockdev --setro /dev/<loopdev>   # after attaching the raw as a loop
device
sudo dd if=/dev/zero of=/dev/<loopdev> bs=512 count=1   # must fail:
Operation not permitted
```

Intended result. An E01 image whose verification hashes match the source, a second independent image on Linux with a matching SHA-256, and a failed write proving the source was protected.

Why this result matters. The image is the evidence you actually analyze; you never touch the original again. Matching verification hashes prove the copy is faithful (Integrity). Two examiners' tools producing the same hash is Reproducibility. The deliberately failed write is how you demonstrate write protection rather than merely claiming it, which is the difference between "I used a write blocker" and "here is proof nothing could be written."

Deliverables. EV03 E01 image, both verification logs, the independent Linux hash, the write-protection test output, and custody lines.

Check your understanding.

- What does image verification actually prove, and what does it not prove?
- Why image the same source twice with different tools?
- How did you demonstrate, not just assert, write protection?

Lab 5: File system analysis with Autopsy and The Sleuth Kit

Objectives. Add the disk image to a forensic case, explore the file system, and recover deleted data.

Principles reinforced. 5 (Reproducibility), 6 (Documentation), 10 (Objectivity).

Prerequisites. EV03 image from Lab 4.

Procedure.

1. **Autopsy on DF-WIN.** New case `DF-2026-001`, store under `02_Working`. Add data source, *Disk Image*, point to the E01. Leave the default ingest modules on (Recent Activity, Hash Lookup, File Type, Keyword Search, Extension Mismatch) and let ingest run.
2. While ingest runs, note how Autopsy reports progress. When done, browse *Data Sources* and the *Views* tree.

3. Find the deleted original. In the file tree, open `Users/dreyes/Desktop` and look for files marked deleted (shown with a red X or under *Deleted Files*). Locate `Q4_Pricing_CONFIDENTIAL.xlsx`.
4. **Command line cross-check on DF-LNX.** Reproduce the same recovery with The Sleuth Kit against your Linux raw copy:

```
mmls /cases/DF-2026-001/02_Working/hpa_lnx.raw          # find the NTFS
partition offset
fls -r -o <offset> /cases/DF-2026-001/02_Working/hpa_lnx.raw | grep -i
pricing
icat -o <offset> /cases/DF-2026-001/02_Working/hpa_lnx.raw <inode> >
/cases/DF-2026-001/03_Exports/recovered.xlsx
sha256sum /cases/DF-2026-001/03_Exports/recovered.xlsx
```

5. Open the recovered spreadsheet and confirm it is the confidential pricing file.

Intended result. Autopsy and The Sleuth Kit both recover the deleted `Q4_Pricing_CONFIDENTIAL.xlsx`, and it opens as the file you seeded.

Why this result matters. Deleting a file usually only removes its directory entry; the data stays in unallocated space until overwritten, which is why recovery works. Finding the same file two ways, with a graphical tool and with command line tools, is Reproducibility in action: an independent examiner can repeat your steps and get your result. Recording the inode and the recovered file's hash is what makes the finding checkable rather than a screenshot.

Deliverables. The Autopsy case, the recovered file with its hash, and notes giving the partition offset and inode.

Check your understanding.

- Why is a deleted file often recoverable?
- What does the SHA-256 of the recovered file let a reviewer do?
- Which tool is "right" if Autopsy and TSK disagree, and how would you resolve it?

Lab 6: Windows artifacts with KAPE and the Eric Zimmerman tools

Objectives. Collect and parse the Windows artifacts that record user activity, and build the story of the file theft from them.

Principles reinforced. 11 (Evidence Corroboration), 6 (Documentation), 10 (Objectivity).

Prerequisites. EV03 image, mounted read-only on DF-WIN.

Procedure.

1. Mount the E01 read-only in FTK Imager (*File, Image Mounting, Block Device / Read Only*). Note the drive letter it gets, for example `G:`.
2. **Collect with KAPE.** Run `gkape.exe` as admin. Target source `G:`, Target `KapeTriage`, target destination `03_Exports\kape_tout`. Module source the same, Module `!EZParser`, module destination `03_Exports\kape_mout`. Execute.
3. Open the parsed CSVs in Timeline Explorer and work through the theft, one artifact per action. For each, record the artifact, the file it lives in, and what it shows:
   - **USB connection:** `SYSTEM` hive, USBSTOR, and `setupapi.dev.log`. Find the TRANSFER drive's first-connected time. Parse the hive with Registry Explorer if you want to see it in place.
   - **File access:** LNK files and Jump Lists (`LECmd` / `JLECmd` output) pointing at `Q4_Pricing_CONFIDENTIAL.xlsx` and `lunch_menu.xlsx`.
   - **Recent documents:** `NTUSER.dat` RecentDocs and the Office MRU for `dreyes`.
   - **Program execution:** Prefetch (`PECmd`) and Amcache (`AmcacheParser`) for the cleaner script's host process and any utilities.
   - **Deletion:** Recycle Bin `$I` records (`RBCmd`) for the deleted original.
4. Build a mini timeline of just these events in a worksheet, each row tagged FACT (what the artifact states) or INTERPRETATION (what you infer).
5. Confirm at least two independent artifacts support the central claim that the pricing file went to the USB drive (for example a LNK file on the USB plus the USBSTOR entry plus the Jump List).

Intended result. A set of parsed artifacts that together show `dreyes` opening the confidential file, renaming it, connecting the TRANSFER drive, and running the cleaner, with the USB theft supported by more than one source.

Why this result matters. No single Windows artifact tells the whole story, and any one of them can be absent or misleading. Corroboration, two or more independent artifacts agreeing, is what turns a plausible reading into a defensible finding. Tagging each row FACT or INTERPRETATION keeps you honest about where the artifact ends and your inference begins, which is Objectivity.

Deliverables. The KAPE output, the parsed CSVs, and the tagged mini timeline worksheet.

Check your understanding.

- Give two independent artifacts that place the pricing file on the USB drive.

- Which of your rows are FACT and which are INTERPRETATION, and why?
- What does a Prefetch entry actually prove about program execution?

Lab 7: Memory forensics with Volatility 3

Objectives. Analyze the memory image from Lab 3 to recover state that never reaches the disk.

Principles reinforced. 7 (Order of Volatility pays off), 11 (Corroboration), 10 (Objectivity).

Prerequisites. EV02 memory image on DF-LNX.

Procedure.

1. Confirm the image hash still matches the value recorded in Lab 3 before analyzing. If it does not, stop and note the integrity failure.
2. Identify the image and list processes:

```
vol -s /opt/tools/vol3-symbols -f /cases/DF-2026-001/01_Evidence/HPA-
WS07_mem.raw windows.info
vol -s /opt/tools/vol3-symbols -f .../HPA-WS07_mem.raw windows.pslist
vol -s /opt/tools/vol3-symbols -f .../HPA-WS07_mem.raw windows.pstree
```

3. Recover command lines and network state:

```
vol ... windows.cmdline      # find the PowerShell command that ran the
cleaner
vol ... windows.netscan      # find the browser's outbound connection to
webmail
```

4. Look for the browser process and the cleaner, and dump a suspicious process if you want to examine it later (`windows.memmap --dump`).
5. Record which findings here also appear on disk (the cleaner in Prefetch from Lab 6) and which exist only in memory (a live network connection).

Intended result. The process list shows the browser and the PowerShell host; `windows.cmdline` shows the command that launched the cleaner; `windows.netscan` shows the outbound webmail connection that no longer exists on disk.

Why this result matters. This is the payoff for capturing memory first in Lab 3. Live network connections and full command lines often exist only in RAM; by shutdown they are gone. Memory findings that also appear on disk corroborate the disk evidence;

memory findings that do not appear on disk are exactly why volatile capture was worth its small footprint. As always, a Volatility plugin's output is a lead: confirm the process is really what it looks like before concluding.

Deliverables. The plugin outputs saved to `03_Exports`, and a note separating memory-only findings from corroborated ones.

Check your understanding.

- Name one finding that existed only in memory. Why was it not on disk?
- How does a memory finding corroborate a disk finding here?
- Why re-hash the memory image before analysis?

Lab 8: Network forensics with Wireshark and Zeek

Objectives. Reconstruct the network side of the incident from the packet capture.

Principles reinforced. 11 (Corroboration), 10 (Objectivity), 6 (Documentation).

Prerequisites. EV04 packet capture (`hpa_capture.pcapng`) from Part 5.

Procedure.

1. **Wireshark on DF-WIN.** Open the capture. Use *Statistics, Conversations* to see who talked to whom. Apply display filters to isolate the webmail traffic, for example `tls.handshake.extensions_server_name contains "mail"` or the destination IP you saw in Lab 7's `netscan`.
2. Follow the TLS stream to the webmail host. You will see that the content is encrypted; record what you can still observe (destination, server name, timing, data volume) and what you cannot (the message body).
3. **Zeek on DF-LNX.** Turn the capture into logs:

```
cd /cases/DF-2026-001/03_Exports && mkdir zeek && cd zeek
zeek -r /cases/DF-2026-001/01_Evidence/hpa_capture.pcapng
cat conn.log | zeek-cut id.orig_h id.resp_h service duration orig_bytes
resp_bytes | head
cat dns.log  | zeek-cut query | sort -u
```

4. From `dns.log` and `conn.log`, identify the webmail domain the subject resolved and connected to, and the size and timing of the upload. Line the upload time up against the file-access time from Lab 6.

Intended result. Zeek's `dns.log` shows the webmail domain being resolved; `conn.log` shows a connection to it with an upstream byte count consistent with sending a file; the timing matches the on-disk file activity.

Why this result matters. Encrypted traffic limits what you can read, and saying so plainly is Objectivity: you report the destination, timing and volume, not an invented message body. The network evidence corroborates the host evidence; the same event seen from two independent vantage points (the endpoint artifacts and the wire) is far stronger than either alone. Zeek is used because reading raw packets by hand does not scale; its logs are the metadata you reason over.

Deliverables. The Zeek logs, the key Wireshark filter and a saved packet or two, and a note matching network timing to host timing.

Check your understanding.

- What can and cannot you learn from the encrypted upload?
- How does the capture corroborate the host artifacts from Lab 6?
- Why convert packets to Zeek logs instead of reading the pcap directly?

Lab 9: Super timelines with Plaso, and reconciling time zones

Objectives. Build one timeline across every artifact source and resolve the time-zone problem that runs through the whole case.

Principles reinforced. 5 (Reproducibility), 10 (Objectivity), 11 (Corroboration).

Prerequisites. EV03 image on DF-LNX.

Procedure.

1. Generate the storage file and export a timeline in UTC with Plaso:

```
cd /cases/DF-2026-001/03_Exports
plaso log2timeline.py --storage-file DF-2026-001.plaso /cases/DF-2026-
001/02_Working/hpa_lnx.raw
plaso pinfo.py DF-2026-001.plaso | head -40
plaso psort.py --output-time-zone UTC -o dynamic -w DF-2026-
001_timeline.csv DF-2026-001.plaso
```

2. Narrow to the window of interest. Re-run psort.py with a date filter around the seeding time, or filter the CSV in Timeline Explorer, to get a readable slice.

3. **Reconcile the clocks.** The subject ran in Central Time; your timeline is in UTC; Lab 3′s response log is in UTC; the packet capture is in UTC. Line up one event you know exactly (the cleaner running, say) across the subject′s displayed local time, the Plaso UTC timeline, and the Zeek logs. Confirm they describe the same moment once converted.
4. Mark the one or two events where a naive reader would misorder things because of the time-zone gap.

Intended result. A single CSV timeline in UTC that interleaves file system, registry, event log and browser events. The theft reads as one ordered sequence, and the Central-to-UTC offset is explicitly resolved.

Why this result matters. A super timeline is the strongest corroboration tool you have, because it forces every source onto one axis, but only if that axis is consistent. Normalizing everything to UTC and documenting the offset is what stops a defensible timeline from becoming a misleading one. Date the observation, not the event: a timestamp records when an artifact says something happened, in some zone, and your job is to state which zone and convert it, not to assume.

Deliverables. The storage file, the UTC timeline CSV, the filtered slice, and the time-zone reconciliation note.

Check your understanding.

- Why export the timeline in UTC rather than the subject′s local time?
- Which event did you use to tie the clocks together, and why that one?
- How could a time-zone error change the apparent order of events?

Lab 10: Anti-forensics detection, timestomping

Objectives. Detect the timestamp manipulation you seeded, by comparing the two sets of NTFS timestamps.

Principles reinforced. 10 (Objectivity), 1 (Integrity), 12 (Defensible Reporting).

Prerequisites. EV03 image on DF-WIN.

Procedure.

1. Parse the master file table from the mounted image with MFTECmd, pointing it at the MFT at the root of the mounted volume, and write the CSV to the exports folder.

2. Open the CSV in Timeline Explorer and find lunch_menu.xlsx. Compare its Standard Information timestamps (the SI columns, what Windows shows) against its File Name timestamps (the FN columns).
3. For a normally created file the two sets are close. For your timestomped file, the SI creation time is three months in the past while the FN creation time holds the real time. Record both.
4. Cross-check with the USN change journal for the rename event, which carries its own timestamp independent of SI.
5. Note that this is detection, not certainty about intent: the mismatch is a fact; "it was done to hide the file" is an interpretation you support with the surrounding activity from Lab 6.

Intended result. lunch_menu.xlsx shows an SI creation time months earlier than its FN creation time, and the change journal shows the real rename time. The manipulation you performed in Part 5 is visible.

Why this result matters. Many anti-forensic techniques leave their own traces. Timestomping usually alters only the easily changed SI timestamps and misses the FN set, so the disagreement between them exposes it. Reporting the mismatch as a fact and the motive as a supported interpretation is Defensible Reporting and Objectivity: you say exactly what the evidence shows and no more. This is also the digital limit of Locard's principle, the manipulation left a trace here, but it might not always.

Deliverables. The MFT CSV, a note with both timestamp sets for the file, and the change-journal cross-check.

Check your understanding.

- Why does timestomping usually miss the File Name timestamps?
- Which part of your conclusion is fact and which is interpretation?
- What would you write if only the SI timestamps were available?

Lab 11: Endpoint hunting with Velociraptor and YARA

Objectives. Use a live-response platform and a YARA rule to find the cleaner utility by its signature, and connect the artifact to adversary behavior.

Principles reinforced. 3 (Forensic Soundness), 11 (Corroboration), 6 (Documentation).

Prerequisites. A clone of HPA-WS07 from SEIZED-STATE on DF-LAB, and the Velociraptor binaries from Part 4.

Procedure.

1. **Start Instant Velociraptor on DF-LNX.** This runs a server and a single local client in one process for learning:

   ```
   velociraptor gui
   ```

   It opens the GUI at `https://127.0.0.1:8889` with username `admin` and password `password`. This mode is for the lab only, not production.
2. Deploy a client on the HPA-WS07 clone. Build an offline collector or run the Windows binary in client mode against your lab server (follow the Velociraptor deployment docs for the exact client config, which the server generates for you).
3. Write a YARA rule that matches the cleaner's unique string and save it as cleaner.yar:

   ```
   rule HPA_Cleaner {
     strings:
       $s = "HPA-CLEANER-SIGNATURE-7731"
     condition:
       $s
   }
   ```

4. Hunt for it. In Velociraptor, collect a YARA process- or file-scan artifact with your rule, or scan the mounted disk directly with the YARA CLI on DF-WIN:

   ```
   yara64.exe -r C:\Cases\DF-2026-001\03_Exports\cleaner.yar G:\Users
   ```

5. Map what you found to MITRE ATT&CK. The cleaner's deletion and log-clearing behavior corresponds to techniques in the Defense Evasion tactic (for example Indicator Removal, T1070). Record the mapping.

Intended result. The YARA rule matches cleanup.ps1 (and its desktop copy); Velociraptor locates the file and related execution evidence; the behavior maps to a named ATT&CK technique.

Why this result matters. Signature hunting scales a single known indicator across a whole endpoint or fleet, which is how real investigations move from one host to many. Finding the same utility through two methods (YARA on disk and Velociraptor collection) is Corroboration. Mapping artifacts to ATT&CK connects "what the file did" to "what tactic it served," which is how technical findings become intelligence a reader can act on. The rule is validated by the fact that you know exactly what it should and should not match.

Deliverables. The YARA rule, the match output, the Velociraptor collection, and the ATT&CK mapping note.

Check your understanding.

- Why hunt with a signature rather than inspecting files one by one?
- What two independent methods confirmed the cleaner, and why does that matter?
- What does an ATT&CK mapping add that a raw artifact does not?

Lab 12: Mobile artifact parsing with ALEAPP

Objectives. Parse a mobile extraction into a readable report and understand the limits of mobile acquisition.

Principles reinforced. 3 (Forensic Soundness), 6 (Documentation), 10 (Objectivity).

Prerequisites. A public Android test image. This lab uses a teaching image rather than the case, because mobile acquisition needs its own device and tooling.

Procedure.

1. Download one of Josh Hickman's public Android images, hosted on Digital Corpora (for example the Android 13 or Android 14 image), together with its documentation. Verify the download hash against the published value.
2. Run ALEAPP against the extraction by launching aleappGUI.exe. Set the input type (tar/zip/fs) to match the download, point it at the extraction, choose an output folder in `03_Exports`, select all artifacts, and process.
3. Open the HTML report. Explore call logs, messaging databases, installed applications, and location artifacts. Pick three findings and confirm each against the image's documentation (the ground truth for this public image).
4. Note what ALEAPP did not or could not recover, and relate it to the acquisition method. A logical acquisition sees less than a full file-system acquisition; encrypted app data may be unreadable.

Intended result. An ALEAPP HTML report whose call, message, app and location findings match the public image's documented contents, plus a short note on acquisition limits.

Why this result matters. Mobile forensics lives in application databases (usually SQLite), and a parser like ALEAPP turns dozens of obscure files into a readable report, which is Documentation at scale. Checking three findings against the image's known contents

validates the tool (Forensic Soundness). Stating what could not be recovered, and why, is Objectivity: the acquisition method sets a hard ceiling on what any tool can find, and an honest report says so rather than implying the absence of data is the absence of activity.

Deliverables. The ALEAPP report, three documentation-confirmed findings, and the acquisition-limits note.

Check your understanding.

- Why is most mobile evidence stored in SQLite databases?
- How did you validate ALEAPP's output?
- How does the acquisition method limit what you can find, and how should a report handle that?

Capstone: full investigation and defensible report

The capstone pulls the twelve labs into one deliverable: a written forensic report on Case DF-2026-001 that would survive review. This is the Presentation and Review pillars, and it is where Defensible Reporting (Principle 12) is graded directly.

What to produce

Write a report (use the template in Appendix E) that answers the scope question from Lab 1: did Dana Reyes exfiltrate the confidential pricing file, and how. The report must include:

1. **Administrative section:** case number, examiner, authority, scope, and the evidence items with their acquisition hashes.
2. **Methodology:** tools and versions used, drawn from your Tool Validation Log, and a one-line statement that each was validated.
3. **Findings:** the exfiltration reconstructed as a UTC timeline, each finding stated as a fact with the artifact and evidence item that supports it, and each key conclusion supported by at least two independent artifacts.
4. **Analysis:** what the facts mean, with interpretations clearly separated from facts and alternative explanations addressed.
5. **Limitations:** what you could not determine and why (encrypted traffic content, acquisition limits, anything the evidence does not settle).
6. **Exhibits:** the timeline, key screenshots or CSV extracts, and the chain of custody.

Peer review (the Review pillar)

Before you submit, exchange reports and one image with a lab partner. Each of you reproduces one finding from the other's report using only what the report documents. If your partner cannot reproduce a finding from your methodology, that finding is not yet defensible; fix the documentation until they can. Record the peer review outcome in both reports.

Mock testimony (optional, recommended)

The instructor plays opposing counsel and challenges three findings: "how do you know the file was on the USB and not just referenced?", "could the timestamp difference be innocent?", "your tool could be wrong, couldn't it?". Defend each with evidence and with your validation record, or concede honestly where the evidence does not support certainty. Conceding what you cannot prove is a strength, not a weakness.

Grading rubric

The full rubric is in Appendix G. In summary, the report is graded on authority and scope (10), chain of custody and integrity across all evidence (15), correct and corroborated findings (25), timeline accuracy including time-zone handling (15), fact-versus-interpretation discipline (15), stated limitations (10), and reproducibility confirmed by peer review (10).

Troubleshooting

Symptom	Likely cause	Fix
Windows 11 installer says the PC does not meet requirements	TPM or Secure Boot not enabled on the VM	Add a TPM and enable Secure Boot in the VM settings (Part 4.1), then reinstall
FTK Imager shows no physical drives	Not running elevated	Close and relaunch with Run as administrator
Autopsy ingest is extremely slow or crashes	JVM memory too low	Raise Maximum JVM memory in Options to 8 GB, restart

Symptom	Likely cause	Fix
`vol` reports no symbol table found	Subject build newer than the symbol pack	Reconnect NAT for that one command so Volatility fetches symbols from Microsoft, then disconnect; note it
`kape.exe --sync` fails	No network, or the KAPE folder is blocked	Connect NAT; run `Get-ChildItem C:\Tools\KAPE -Recurse
EZ Tools GUI will not start	.NET Desktop Runtime 9 missing	Install it with `winget install Microsoft.DotNet.DesktopRuntime.9`
Ghidra will not launch	Wrong Java, or a JRE instead of a JDK	Point Ghidra at the JDK 21 folder, not a JRE; confirm with `java -version`
`zeek: command not found`	PATH not updated	`echo 'export PATH=$PATH:/opt/zeek/bin' >> ~/.bashrc && source ~/.bashrc`
`docker` permission denied	Account not in the docker group yet	`sudo usermod -aG docker $USER`, then log out and back in
Plaso run fails with a permissions error on output	Container writes as root	`sudo chown $USER /cases/DF-2026-001/03_Exports/*.plaso` afterwards
DF-LNX cannot reach the DF-WIN share	Firewall or wrong credentials	Confirm both are on DF-LAB, allow File and Printer Sharing on DF-WIN, re-mount with the examiner credentials
`ping` to DF-WIN fails	Windows Firewall blocks ICMP	`Enable-NetFirewallRule -Name FPS-ICMP4-ERQ-In` on DF-WIN
Hashes do not match after copying evidence	Copy error, or you compared different algorithms	Re-copy, compare the same algorithm, and check the source was read-only
Timeline events look an hour or more off	Time-zone confusion	Re-export with `--output-time-zone UTC` and redo the reconciliation in Lab 9

Appendices

Appendix A: Chain of custody and authorization form

Authorization record (one per case)

Field	Entry
Case number	DF-2026-001
Authorized by (name, title)	
Authority type	Consent / Policy / Warrant / Other
Scope (what may be examined)	
Out of scope (what may not)	
Date authorized	
Examiner	

Chain of custody (one per evidence item)

Field	Entry
Evidence ID	DF-2026-001-EV__
Description	
Acquisition hash (SHA-256)	

Date/time (UTC)	Action	From	To	Purpose	Signature
	Received				

Appendix B: Evidence register

Evidence ID	Description	Type	Acquired (UTC)	SHA-256	Location
DF-2026-001-EV01	TRANSFER USB drive	Physical			01_Evidence
DF-2026-001-EV02	HPA-WS07 memory image	Raw image			01_Evidence

Evidence ID	Description	Type	Acquired (UTC)	SHA-256	Location
DF-2026-001-EV03	HPA-WS07 disk image	E01			01_Evidence
DF-2026-001-EV04	Network capture	pcapng			01_Evidence

Appendix C: Case notes template

One file per examiner in `04_Notes`. Every entry is contemporaneous; if you write it up later, say so and give the delay.

```
Case: DF-2026-001        Examiner: ____________
Session start (UTC): ____________

[UTC time]  Action taken
[UTC time]  Tool + version used, exact command
[UTC time]  Result observed (FACT)
[UTC time]  What I infer from it (INTERPRETATION)
[UTC time]  Hash recorded / custody updated
...
Session end (UTC): ____________
```

Appendix D: Command reference

Task	Windows	Linux
SHA-256 of a file	`Get-FileHash -Algorithm SHA256 f`	`sha256sum f`
Current UTC time	`(Get-Date).ToUniversalTime().ToString('u')`	`date -u '+%F %T UTC'`
Image a disk	FTK Imager, Create Disk Image, E01	`dc3dd if=/dev/sdX of=out.raw hash=sha256 log=l.log`
Verify an E01	FTK Imager verify	`ewfverify image.E01`
List partitions	`Get-Disk`, `Get-Partition`	`mmls image.raw`
List/recover files in an image	Autopsy	`fls -r -o OFF img`, `icat -o OFF img INODE`

Task	Windows	Linux
Memory: processes	(Volatility on DF-LNX)	`vol -s SYM -f mem.raw windows.pslist`
Memory: command lines		`vol ... windows.cmdline`
Memory: network		`vol ... windows.netscan`
Parse MFT	`MFTECmd.exe -f MFT --csv out`	
Parse event logs	`EvtxECmd.exe -d dir --csv out`	
Build super timeline		`plaso log2timeline.py --storage-file c.plaso src`
Export timeline (UTC)		`plaso psort.py --output-time-zone UTC -o dynamic -w t.csv c.plaso`
pcap to logs		`zeek -r capture.pcapng`
Read a Zeek log column		`cat conn.log
Scan with YARA	`yara64.exe -r rule.yar path`	`yara -r rule.yar path`
Read metadata	`exiftool file`	`exiftool file`

Appendix E: Report template

```
FORENSIC EXAMINATION REPORT

1. Administrative
   Case number, examiner, dates
   Authority and scope (from Lab 1)
   Evidence items and acquisition hashes (Appendix B)
```

```
2. Methodology
   Tools and versions (from the Tool Validation Log)
   Statement that each tool was validated

3. Findings
   Each finding: what, which artifact, which evidence item, UTC time
   Key conclusions corroborated by two or more independent artifacts
   A UTC timeline of the incident

4. Analysis
   What the facts mean; interpretations labelled as such
   Alternative explanations considered

5. Limitations
   What could not be determined and why

6. Exhibits
   Timeline, extracts, chain of custody

7. Examiner statement and signature
```

Appendix F: Tool validation log

Tool	Version	Source URL	Installer SHA-256	Reference dataset	Expected result	Obtained result	Pass?	Date

Appendix G: Capstone grading rubric

Criterion	Points	What earns full marks
Authority and scope	10	Authorization recorded, scope and out-of-scope stated, work stays within scope
Chain of custody and integrity	15	Every evidence item has custody from acquisition, acquisition hashes recorded and verified
Findings correct and corroborated	25	Exfiltration reconstructed accurately; each key conclusion rests on two or more independent artifacts

Criterion	Points	What earns full marks
Timeline and time zones	15	Single UTC timeline; Central-to-UTC offset resolved and documented
Fact vs interpretation	15	Facts and interpretations clearly separated throughout; no overstatement
Limitations	10	Honest account of what could not be determined and why
Reproducibility	10	A partner reproduced a finding from the report alone; peer review recorded

Appendix H: Standards quick reference

Reference	One-line scope
NIST SP 800-86	Integrating forensic techniques into incident response; collection, examination, analysis, reporting
NIST SP 800-61 Rev. 3	Incident response aligned to the Cybersecurity Framework 2.0
NIST SP 800-101 Rev. 1	Mobile device forensics
ISO/IEC 27037	Identification, collection, acquisition, preservation of digital evidence
ISO/IEC 27041 / 27042 / 27043	Method suitability / analysis and interpretation / investigation process
ISO/IEC 17025	Laboratory competence
SWGDE best practices	Discipline-specific procedures, validation, reporting
Federal Rules of Evidence 702, 901, 902; Daubert	Expert testimony, authentication, scientific reliability (US)
MITRE ATT&CK	Mapping artifacts to adversary tactics and techniques